FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

PROGRAMME: BSSE-M

YEAR/SEM: YEAR 2 SEM 1

COURSE CODE: BIT2115

PROBLEM NAME: inheritance and PolyMorphism

STUDENT DETAILS:

Name of Student	**Amiiro Alisaid Ali**
Reg Number	**24/657/BSSE-M**

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Table of contents :

Q1……..

Line of the code Main class and explanation ……5

Detailed screenshot of Main calss……6

Line of the code Animal class and explanation ……..7

Detailed screenshot of Animal calss…….8

Line of the code Cat class and explanation …….9

Detailed screenshot of Cat calss……..10

Line of the code Dog class and explanation …….11

Detailed screenshot of Dog calss……..12

Line of the code Poppy class and explanation ……13

Detailed screenshot of Poppy calss……14

The output……15

Q2……..

Line of the code Main class and explanation ……16

Detailed screenshot of Main calss……17

Line of the code Entity class and explanation ……..18

Detailed screenshot of Entity calss…….19

Line of the code Human class and explanation ……20

Detailed screenshot of Human calss……21

Line of the code Robot class and explanation …….22

Detailed screenshot of Robot calss……..23

Line of the cod PolymorphismTester class and explanation ……..…24

Detailed screenshot of PolymorphismTester calss……25

The output……..26

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Figure 1 main class 5
Figure 2 the parent class Animal 7
Figure 3 subclass Animal 9
Figure 4 Subclass Animal 11
Figure 5 of subclass Puppy class 13
Figure 6 the output 14
Figure 7 the main class 17
Figure 8 Subclass Human 19
Figure 9 Parent class Entity 21
Figure 10 Subclass Robot 23
Figure 11 tester class 25
Figure 12 the output 26

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Code and Line explanation :

Line	Explanation
Line1	Declares the main class named Main.
Line 2	The entry point — Java starts running your program from here.
Line 3	Just prints a label to the console so you know which section is running.
Line4	Creates a Dog object. Passes name "Rexx", age 4, and breed "German Shepherd" to the Dog constructor.
Line 5	Calls Dog's overridden describe() — prints Dog-specific info (name, age, breed). It uses Dog's version, not Animal's.
Line 6	Dog has no eat() of its own, so Java goes up to Animal and runs Animal's eat(). This is inheritance in action.
Line 7	Calls bark() — a method that only Dog has. Animal doesn't have it.
Line 8	Prints a label for the next section.
Line 9	Creates a Puppy object with name, age, breed, and favourite toy. Puppy's constructor calls Dog's constructor, which calls Animal's — a chain of super() calls.
Line 10	Calls Puppy's overridden describe() — shows Puppy-specific info including the toy.
Line 11	Puppy has no eat(), Dog has no eat(), so Java goes all the way up to Animal to find it. Three levels up.
Line 12	Puppy has no bark(), so Java finds it one level up in Dog and runs it.
Line 13	Calls play() — a method unique to Puppy. No other class has it.
Line 14	Prints a label. "Hierarchical" means both Cat and Dog extend Animal separately — two branches from one root.
Line 15	Creates a Cat object. true means the cat is an indoor cat.
Line 16	Calls Cat's overridden describe() — shows Cat-specific info (name, age, indoor status).
Line 17	Cat has no eat(), so Java goes up to Animal and runs Animal's version. Same as Dog did — both share the same parent.
Line 18	Calls purr() — a method unique to Cat. Just like bark() belongs only to Dog.
Line 19 , line 20	Closes the main method and the Main class.

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Figure1 Main class:

Current File

Main.java | Animal.java | Cat.java | Dog.java | Poppy.java

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("SINGLE INHERITANCE (Dog -> Animal)");
        Dog dog = new Dog( name: "Rexx",  age: 4,  breed: "German Shepherd");
        dog.describe();
        dog.eat();
        dog.bark();
        System.out.println("MULTILEVEL INHERITANCE (Puppy -> Dog ->Animal) ");
        Puppy puppy = new Puppy( name: "Buddy",  age: 1,  breed: "Labrador",  toyName: "rubber ball");
        puppy.describe();
        puppy.eat();
        puppy.bark();
        puppy.play();
        System.out.println(" HIERARCHICAL INHERITANCE (Cat -> Animal, Dog → Animal) ");
        Cat cat = new Cat( name: "Yara",  age: 3,  isIndoor: true);
        cat.describe();
        cat.eat();
        cat.purr();
    }
}
```

Figure 1 main class

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Code and line Explanation:

Line	Code Exaplanation
Line 1	Declares the class named Animal. This is the base class — all other classes will inherit from it.
Line 2	Declares a String attribute called name. Every animal will have a name.
Lline 3	Declares an int attribute called age. Every animal will have an age.
Line 4	The constructor — called when you create an Animal object. It accepts name and age as inputs.
Line 5	this.name = name — stores the passed-in name into the object's own name attribute.
Line 6	this.age = age — stores the passed-in age into the object's own age attribute. } — closes the constructor.
Line 7	} — closes the constructor.
Line 8	Declares the eat() method — any animal can call this.
Line 9	Prints the animal's name followed by " is eating."
Line 10	} — closes the eat() method.
Line 11	Declares the describe() method — subclasses will override this with their own version.
Line 12	Prints the animal's name and age to the console.
Line 13	} — closes the describe() method.
Line 14	} — closes the Animal class.

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Detailed screenshot of Animal class:

Current File

Main.java | Animal.java | Cat.java | Dog.java | Poppy.java

```java
class Animal {
    String name;
    int age;
    Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }
    void eat() {
        System.out.println(name + " is eating.");
    }
    void describe() {
        System.out.println("Animal: " + name + ", Age: " + age);
    }
}
```

Figure 2 the parent class Animal

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Code and line explanation :

Lines	Code Explanation
Line1	Declares the Cat class. extends Animal means Cat inherits all attributes and methods from Animal — this is hierarchical inheritance.
Line 2	Declares a boolean attribute called isIndoor. This is unique to Cat — Animal and Dog don't have it.
Line 3	The constructor — called when you create a Cat object. It accepts name, age, and isIndoor as inputs.
Line 4	super(name, age) — calls Animal's constructor to set the name and age attributes. Must always be the first line in the constructor.
Line 5	this.isIndoor = isIndoor — stores the passed-in value into the Cat's own isIndoor attribute.
Line 6	} — closes the constructor.
Line 7	@Override — tells Java that the method below is intentionally replacing Animal's version of describe().
Line 8	Declares the describe() method — this is Cat's own version, overriding Animal's.
Line 9	Prints Cat-specific info: name, age — first part of the output (continues on next line).
Line 10	Continues the print statement, adding isIndoor to the output. Split across two lines for readability.
Line 11	} — closes the describe() method.
Line 12	Declares purr() — a method unique to Cat. No other class has this.
Line 13	Prints the cat's name followed by " says: Purrrr..." to the console.
Line 14	} — closes the purr() method
Line 15	} — closes the Cat class.

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Detailed screenshot of Cat Class:

Main.java | Animal.java | Cat.java | Dog.java | Poppy.java

```java
class Cat extends Animal {
    boolean isIndoor;
    Cat(String name, int age, boolean isIndoor) {
        super(name, age);
        this.isIndoor = isIndoor;
    }
    @Override
    void describe() {
        System.out.println("Cat: " + name + ", Age: " + age +
                ", Indoor: " + isIndoor);
    }
    void purr() {
        System.out.println(name + " says: Purrrr...");
    }
}
```

Figure 3 subclass Animal

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Code and line Explanation:

Code	Explanation
Line 1	Declares the Dog class. extends Animal means Dog inherits all attributes and methods from Animal — this is single inheritance.
Line 2	Declares a String attribute called breed. This is unique to Dog — Animal and Cat don't have it.
Line 3	The constructor — called when you create a Dog object. It accepts name, age, and breed as inputs.
Line 4	super(name, age) — calls Animal's constructor to handle setting name and age. The comment explains this. Must always be the first line in the constructor.
Line 5	this.breed = breed — stores the passed-in breed into the Dog's own breed attribute.
Line 6	} — closes the constructor.
Line 7	@Override — tells Java that the method below is intentionally replacing Animal's version of describe().
Line 8	8Declares the describe() method — this is Dog's own version, overriding Animal's.
Line 9	Prints Dog-specific info: name, age, and breed all on one line to the console.
Line 10	} — closes the describe() method.
Line 11	Declares bark() — a method unique to Dog. No other class has this.
Line 12	Prints the dog's name followed by " says: Woof!" to the console.
Line 13	} — closes the bark() method and the Dog class.

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Detailed Screenshot of Dog class:

Main.java | Animal.java | Cat.java | Dog.java | Poppy.java

```java
class Dog extends Animal {
    String breed;
    Dog(String name, int age, String breed) {
        super(name, age);        // calls Animal constructor
        this.breed = breed;
    }
    @Override
    void describe() {
        System.out.println("Dog: " + name + ", Age: " + age + ", Breed: " + breed);
    }
    void bark() {
        System.out.println(name + " says: Woof!");
    }
}
```

Figure 4 Subclass Animal

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Line and code Explanation:

Line	Explanation
Line 1	Declares the Puppy class. extends Dog means Puppy inherits from Dog, which itself inherits from Animal — this is multilevel inheritance.
Line 2	Declares a String attribute called toyName. This is unique to Puppy — no other class has it.
Line 3	The constructor — called when you create a Puppy object. It accepts name, age, breed, and toyName as inputs.
Line 4	super(name, age, breed) — calls Dog's constructor, which then calls Animal's constructor. A chain of super() calls goes up two levels.
Line 5	this.toyName = toyName — stores the passed-in toy name into Puppy's own toyName attribute.
Line 6	} — closes the constructor.
Line 7	Declares play() — a method unique to Puppy. No other class has this.
Line 8	Prints the puppy's name and its favourite toy to the console.
Line 9	} — closes the play() method.
Line 10	@Override — tells Java the method below is intentionally replacing Animal's version of describe().
Line 11	Declares the describe() method — this is Puppy's own version, overriding Animal's.
Line 12	Prints Puppy-specific info: name and age — first part of the output (continues on next line).
Line 13	Continues the print statement, adding breed and toyName to the output. Split across two lines for readability.
Line 14	} — closes the describe() method.
Line 15	} — closes the Puppy class.

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Detailed screenshot of Poppy class:

Current File

Main.java | Animal.java | Cat.java | Dog.java | Poppy.java

```java
class Puppy extends Dog {
    String toyName;
    Puppy(String name, int age, String breed, String toyName) {
        super(name, age, breed);
        this.toyName = toyName;
    }
    void play() {
        System.out.println(name + " is playing with its " + toyName + "!");
    }
    @Override
    void describe() {
        System.out.println("Puppy: " + name + ", Age: " + age +
                ", Breed: " + breed + ", Favourite toy: " + toyName);
    }
}
```

Figure 5 of subclass Puppy class

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

The output:

```
Run    Main
"C:\Program Files\Java\jdk-18.0.2\bin\java.exe" "-javaagent:C:\Program Files
SINGLE INHERITANCE (Dog -> Animal)
Dog: Rexx, Age: 4, Breed: German Shepherd
Rexx is eating.
Rexx says: Woof!
MULTILEVEL INHERITANCE (Puppy -> Dog ->Animal)
Puppy: Buddy, Age: 1, Breed: Labrador, Favourite toy: rubber ball
Buddy is eating.
Buddy says: Woof!
Buddy is playing with its rubber ball!
 HIERARCHICAL INHERITANCE (Cat -> Animal, Dog → Animal)
Cat: Yara, Age: 3, Indoor: true
Yara is eating.
Yara says: Purrrr...

Process finished with exit code 0
```

Figure 6 the output

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Key Takeaways:

1. Multilevel Inheritance Puppy extends Dog, Dog extends Animal — a 3-level chain.

2. super() goes up the chain super(name, age, breed) calls Dog's constructor, which automatically calls Animal's constructor.

3. Puppy reuses without rewriting Puppy never writes eat() or bark() — it just borrows them from Dog and Animal.

4. @Override replaces the parent's version Puppy has its own describe() so that one runs instead of Dog's or Animal's.

5. Puppy adds new things toyName and play() are unique to Puppy — no other class has them.

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Q2

Line and code Explanation :

Line	Code Explanation
Line 1	Declares the main class named Main.
Line 2	The entry point — Java starts running the program from here.
Line 3	Creates a Robot object and passes "R2-D2" as the name.
Line 4	Creates a Human object and passes "Amira" as the name.
Line 5	Prints a label so you know the overriding section is running.
Line 6	Calls performAction() on robot — runs Robot's overridden version.
Line 7	Calls performAction() on human — runs Human's overridden version.
Line 8	Prints a label so you know the overloading section is running.
Line 9	Calls displayDetails() with no argument — runs the no-parameter version.
Line 10	Calls displayDetails() with "Model: X100" — runs the parameterised version.
Line 11	Calls displayDetails() with no argument on human — runs the no-parameter version.
Line 12	Calls displayDetails() with "Occupation: Engineer" — runs the parameterised version.
Line 13	Prints a label so you know the runtime polymorphism section is running.
Line 14	Passes robot to triggerAction() — JVM decides at runtime to run Robot's performAction().
Line 15	Passes human to triggerAction() — JVM decides at runtime to run Human's performAction().
Line 16	} — closes the main method.
Line 17	} — closes the Main class.

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Detailed ScreenShot of main class:

Main.java × | Human.java | Entity.java | Robot.java | PolymorphismTester.java

```java
public class Main {
    public static void main(String[] args) {
        Robot robot = new Robot( name: "R2-D2");
        Human human = new Human( name: "Amira");
        System.out.println("  METHOD OVERRIDING ");
        robot.performAction();
        human.performAction();
        System.out.println(" METHOD OVERLOADING ");
        robot.displayDetails();
        robot.displayDetails( extraInfo: "Model: X100");
        human.displayDetails();
        human.displayDetails( extraInfo: "Occupation: Engineer");
        System.out.println("RUNTIME POLYMORPHISM");
        PolymorphismTester.triggerAction(robot);
        PolymorphismTester.triggerAction(human);
    }
}
```

Figure 7 the main class

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Code and Line of Explanation of Human class

Line	Explanation
Line 1	Declares the Human class. extends Entity means Human inherits all attributes and methods from Entity.
Line 2	The constructor — called when you create a Human object. It accepts name as input.
Line 3	super(name) — calls Entity's constructor to set the entityName attribute. Must always be the first line in the constructor.
Line 4	} — closes the constructor.
Line 5	@Override — tells Java the method below is intentionally replacing Entity's version of performAction().
Line 6	Declares the performAction() method — this is Human's own version, overriding Entity's.
Line 7	Prints the human's name followed by " (Human) thinks and makes a decision." to the console.
Line 8	} — closes the performAction() method.
Line 9	} — closes the Human class.

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Detailed screeShot of human class:

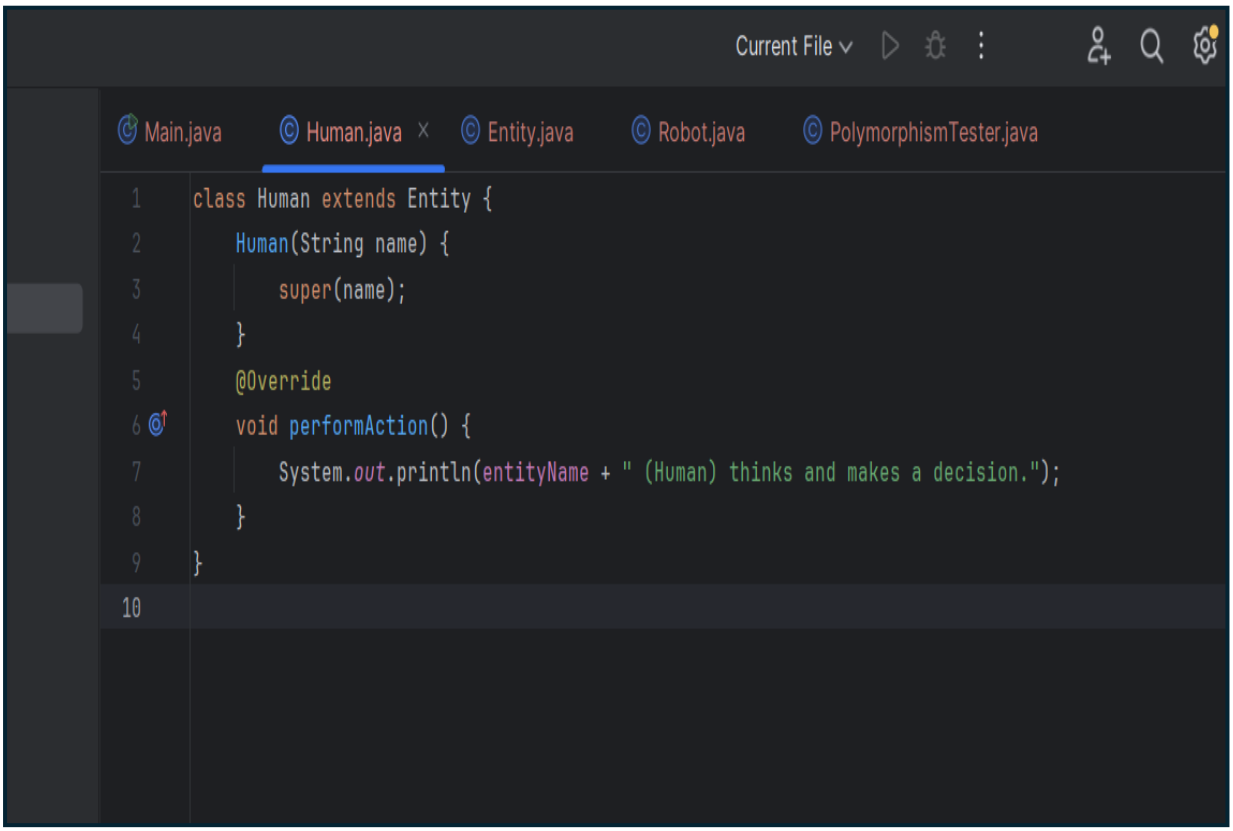

Figure 8 Subclass Human

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Line and code Explanation :

Line	Explanation
Line 1	Declares the Entity class. This is the base class — Robot and Human will inherit from it.
Line 2	Declares a String attribute called entityName. Every entity will have a name.
Line 3	The constructor — called when you create an Entity object. It accepts entityName as input.
Line 4	this.entityName = entityName — stores the passed-in name into the object's own entityName attribute.
Line 5	} — closes the constructor.
Line 6	Declares performAction() — a general action method that subclasses will override with their own version.
Line 7	Prints the entity's name followed by " performs a generic action." to the console.
Line 8	} — closes the performAction() method.
Line 9	Declares the first version of displayDetails() — takes no parameters. This is method overloading.
Line 10	Prints a default message showing just the entity's name.
Line 11	} — closes the first displayDetails() method.
Line 12	Declares the second version of displayDetails() — takes a String parameter called extraInfo. This is method overloading.
Line 13	Prints the entity's name together with the extra information passed in.
Line 14	} — closes the second displayDetails() method and the Entity class.

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Detailed screeShot of Entity class:

Main.java | Human.java | Entity.java | Robot.java | PolymorphismTester.java

```java
class Entity {
    String entityName;
    Entity(String entityName) {
        this.entityName = entityName;
    }
    void performAction() {
        System.out.println(entityName + " performs a generic action.");
    }
    void displayDetails() {
        System.out.println("[Default] Entity name: " + entityName);
    }
    void displayDetails(String extraInfo) {
        System.out.println("[Details] Entity: " + entityName + " | Info: " + extraInfo);
    }
}
```

Figure 9 Parent class Entity

INTERNATIONAL UNIVERSITY OF EAST AFRICA
'Learning to Succeed'

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Line and code Explanation of Robot class:

Line	Explanation
Line 1	Declares the Robot class. extends Entity means Robot inherits all attributes and methods from Entity.
Line 2	The constructor — called when you create a Robot object. It accepts name as input.
Line 3	super(name) — calls Entity's constructor to set the entityName attribute. Must always be the first line in the constructor.
Line 4	} — closes the constructor.
Line 5	@Override — tells Java the method below is intentionally replacing Entity's version of performAction().
Line 6	Declares the performAction() method — this is Robot's own version, overriding Entity's.
Line 7	Prints the robot's name followed by " (Robot) executes a programmed task." to the console.
Line 8	} — closes the performAction() method and the Robot class.

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Detailed ScreenShot of Robot class:

Current File

Main.java | Human.java | Entity.java | Robot.java | PolymorphismTester.java

```java
class Robot extends Entity {
    Robot(String name) {
        super(name);
    }
    @Override
    void performAction() {
        System.out.println(entityName + " (Robot) executes a programmed task.");
    }
}
```

Figure 10 Subclass Robot

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Line and Code Explanation:

Line	Explanation
Line 1	Declares the PolymorphismTester class. Its only job is to test runtime polymorphism.
Line 2	Declares a static method called triggerAction(). It accepts an Entity reference as input — meaning you can pass a Robot or Human object into it.
Line 3	Calls performAction() on whatever object was passed in. The JVM decides at runtime which version to run — Robot's or Human's.
Line 4	} — closes the method and the class.

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Detailed ScreenShot of PolymorphismTester:

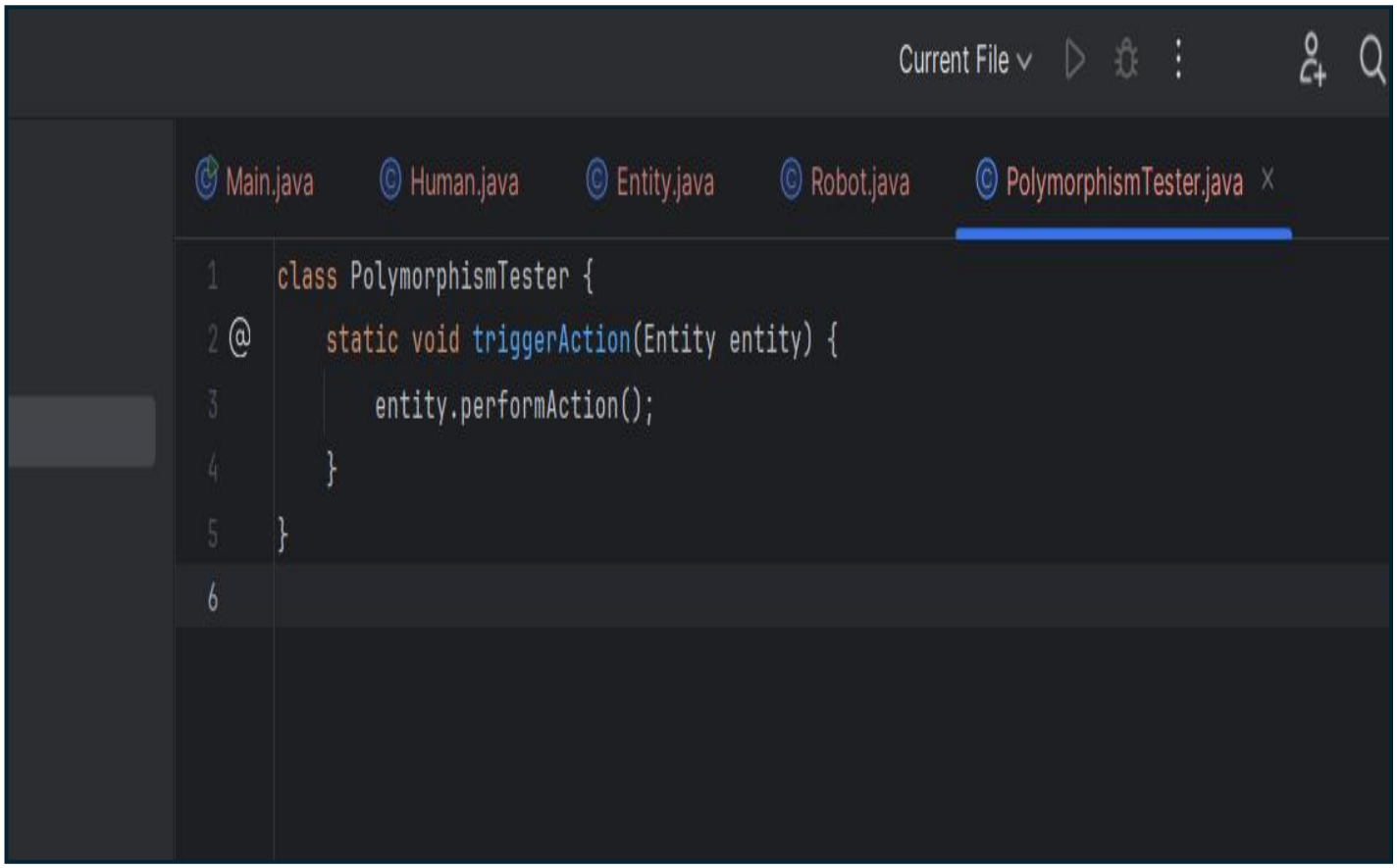

Figure 11 tester class

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Output:

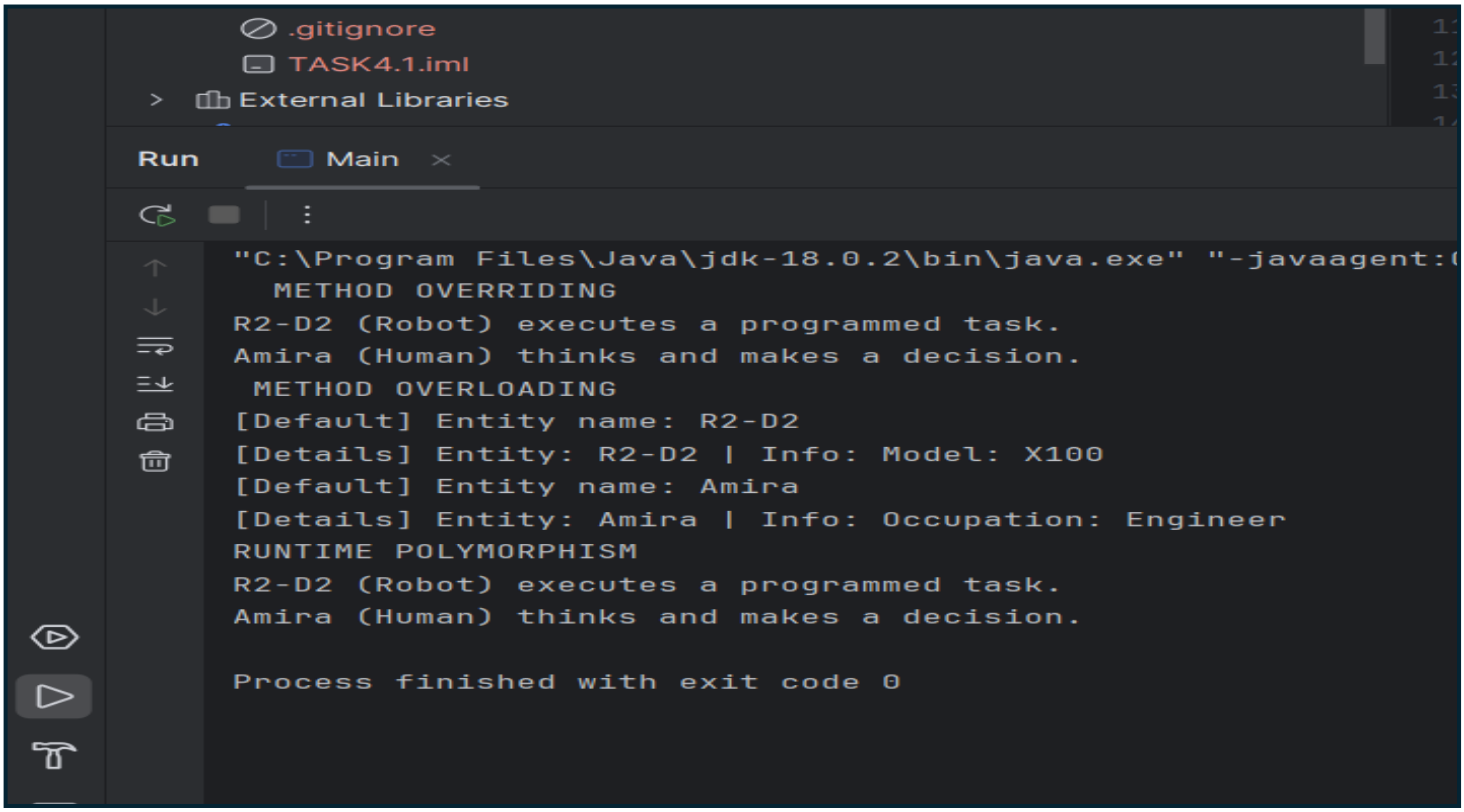

Figure 12 the output

FACULTY OF SCIENCE AND TECHNOLOGY
LOGIC BUILDING & PROGRAMMING REPORT
APPLIED OBJECT-ORIENTED PROGRAMMING CLASS ACTIVITY

Key takeaways:

1. static — means you call it directly on the class without creating an object.
2. Entity entity — accepts any object that extends Entity, so Robot and Human both work.
3. The JVM decides at runtime which performAction() to run — Robot's or Human's.
4. Same method call, different output depending on the object passed — this is runtime polymorphism.